

Rapport Technique

Support des formats 3D et système LOD
dans Babylon.js v8.50.2

Date: 09/02/2026

Table des matières

Table des matières.....	2
1. Introduction.....	2
2. Support du Gaussian Splatting (.PLY).....	3
2.1 Vue d'ensemble.....	3
2.2 Formats supportés.....	3
2.3 Schéma PLY requis.....	3
2.4 Caractéristiques techniques.....	3
3. Support des fichiers .OBJ.....	4
3.1 Capacités du loader OBJ.....	4
3.2 Configuration et options importantes.....	4
3.3 Support des textures.....	4
3.3.1 Limitation critique: Format EXR.....	5
4. Système Level of Detail (LOD).....	5
4.1 Architecture du système LOD.....	5
4.1.1 Approche par fichiers multiples.....	5
4.1.2 Auto-LOD: Simplification automatique.....	5
4.2 Gestion des textures avec LOD.....	6
4.3 Recommandations pour l'utilisation du LOD.....	6
5. Limitations techniques identifiées.....	6
5.1 Format EXR.....	6
5.2 LOD et précision des UV.....	6
5.3 Architecture du système LOD.....	7
5.4 Performance et complexité.....	7
6. Recommandations.....	7
6.1 Pour le support des textures .EXR.....	7
6.2 Pour le système LOD avec .OBJ.....	7
6.3 Stratégie globale.....	7
7. Conclusion.....	8
8. Références.....	8

1. Introduction

Ce rapport analyse les capacités de Babylon.js version 8.50.2 concernant le support des formats 3D, en particulier les fichiers .PLY (Gaussian Splatting), .OBJ, et l'implémentation du système Level of Detail (LOD). L'objectif est d'identifier les limites techniques de Babylon.js pour notre cas d'usage spécifique impliquant des fichiers .OBJ haute résolution avec textures .EXR et système LOD.

2. Support du Gaussian Splatting (.PLY)

2.1 Vue d'ensemble

Babylon.js 8.0 a introduit le support natif du Gaussian Splatting, une technique de rendu point-based qui représente des objets 3D comme une collection d'ellipsoïdes gaussiens orientés en 3D. Cette technologie permet un rendu haute qualité de scènes 3D complexes avec une efficacité computationnelle remarquable.

2.2 Formats supportés

Le loader SPLAT de Babylon.js gère plusieurs formats de fichiers:

- .splat - Format standard non compressé
- .ply - Format PLY avec schéma Gaussian Splatting spécifique
- .spz - Format SuperSplat compressé (ajouté dans v8.0)

2.3 Schéma PLY requis

Important: Les fichiers .PLY pour Gaussian Splatting nécessitent un schéma très spécifique. Un simple nuage de points (x, y, z, r, g, b) ne fonctionnera pas. Le format doit inclure:

- Position (x, y, z)
- Normales (nx, ny, nz)
- Coefficients Spherical Harmonics (f_dc_0 à f_dc_2, f_rest_0 à f_rest_44)
- Opacité (encodée comme $\log(\alpha / (1 - \alpha))$)
- Échelles (scale_0, scale_1, scale_2)
- Rotation quaternion (rot_0, rot_1, rot_2, rot_3)

2.4 Caractéristiques techniques

Caractéristique	Description
Stockage GPU	Textures statiques optimisées (v8.0+) au lieu de buffers d'instances dynamiques
Performance	60 FPS pour des modèles de 2.5 millions de splats (exemple: modèle 'truck')
Tri	Tri $O(n \log n)$ des splats par distance à la caméra

Exigences	WebGL 2 ou WebGPU recommandé pour les meilleures performances
Mémoire	Utilisation importante de la mémoire GPU pour le stockage basé sur les textures

3. Support des fichiers .OBJ

3.1 Capacités du loader OBJ

Le loader OBJ de Babylon.js (OBJFileLoader) offre un support complet du format Wavefront OBJ avec les fonctionnalités suivantes:

- Objets/groupes
- Faces: triangles, quads, polygones
- Couleurs: diffuse, ambient, specular, alpha
- Textures via fichiers MTL: ambient, diffuse, specular, bump, opacity
- Multi-matériaux (création de sous-meshes)
- Couleurs de vertex (si présentes dans le fichier)

3.2 Configuration et options importantes

Plusieurs options de configuration sont disponibles pour optimiser le chargement:

Option	Description
OPTIMIZE_WITH_UV	Active l'optimisation des UV pour éviter les distorsions de texture (temps de chargement plus long)
OPTIMIZE_NORMALS	Améliore les discontinuités de shading lors de l'utilisation de OPTIMIZE_WITH_UV
INVERT_TEXTURE_Y	Inverse l'axe Y des coordonnées UV (correction selon le logiciel d'export)
MATERIAL_LOADING_FAILS_SILENTLY	false = force le chargement strict des matériaux avec callbacks d'erreur
SKIP_MATERIALS	Ignore le fichier MTL même s'il est défini dans l'OBJ

3.3 Support des textures

Les textures sont gérées via les fichiers MTL (Material Template Library). Babylon.js supporte:

- Formats d'image: JPG, PNG, JPEG, BMP (tous les formats supportés par le navigateur)
- Contrôle UV: uOffset, vOffset, uScale, vScale
- Canal alpha

3.3.1 Limitation critique: Format EXR

Point important: Babylon.js ne supporte pas nativement les fichiers .EXR (OpenEXR) comme textures pour les meshes standards. Le support EXR/HDR est limité aux:

- Textures d'environnement HDR (cube maps)
- Fichiers .env (format propriétaire Babylon pour l'IBL)
- Fichiers .hdr convertis en cube textures

Solution recommandée: Pour utiliser des textures haute résolution avec nos fichiers .OBJ, il faudra convertir les fichiers .EXR vers un format compatible (PNG 16-bit, TIFF) avant le chargement. Les performances et la qualité dépendront de la résolution finale et de la compression choisie.

4. Système Level of Detail (LOD)

4.1 Architecture du système LOD

Le système LOD de Babylon.js fonctionne selon le principe de **multiples meshes**, contrairement à certains formats comme Nexus (.nxz) qui utilisent un fichier unique avec décimation dynamique. Voici comment cela fonctionne:

4.1.1 Approche par fichiers multiples

Le développeur doit:

- Créer manuellement plusieurs versions du mesh à différents niveaux de détail
- Charger ces meshes dans la scène
- Les associer au mesh principal via addLODLevel()
- Définir les distances de transition

4.1.2 Auto-LOD: Simplification automatique

Babylon.js offre une fonctionnalité d'Auto-LOD qui peut **générer automatiquement** les niveaux de détail à partir d'un seul mesh haute résolution. Cette approche se rapproche de notre hypothèse:

Fonctionnement:

- Charge un seul fichier haute résolution
- Utilise un algorithme de simplification quadratique (Quadratic Error Simplification)
- Génère automatiquement des versions simplifiées en mémoire
- Traitement asynchrone pour ne pas bloquer le rendu
- Ajoute automatiquement les niveaux LOD à la scène

Exemple de code:

```
mesh.simplify([
  { quality: 0.9, distance: 25 },
  { quality: 0.5, distance: 50 },
  { quality: 0.3, distance: 100 }
]);
```

4.2 Gestion des textures avec LOD

Point critique pour notre cas d'usage: Lors de la simplification automatique, les coordonnées UV peuvent être légèrement décalées. L'algorithme se concentre sur la position des vertices uniquement (pour des raisons de performance) et ne recalcule pas les normales, couleurs ou coordonnées UV de manière optimale.

Conséquences:

- Les UVs sont généralement "assez bons" mais pas parfaits
- Problème mineur pour des textures à basse/moyenne résolution
- **Potentiellement problématique pour des textures .EXR très haute définition**
- Les matériaux restent les mêmes sur tous les niveaux LOD (même texture appliquée)

4.3 Recommandations pour l'utilisation du LOD

Selon la documentation Babylon.js, voici les bonnes pratiques:

- Utiliser pour des meshes de plus de 500 faces (en dessous, peu d'intérêt)
- Meilleur résultat avec des objets complexes et cohérents
- Éviter la simplification de meshes contenant plusieurs objets distants dans un seul mesh
- Les distances doivent être non-linéaires (qualité 90% peut être proche, mais qualité 30% doit être loin)
- L'optimisation du mesh (optimizeMesh: true) peut éviter les trous dans la géométrie

5. Limitations techniques identifiées

5.1 Format EXR

- **Babylon.js v8.50.2 ne supporte pas nativement les fichiers .EXR comme textures de mesh**
- Le support HDR/EXR est limité aux environnements (cube maps, IBL)
- Nécessite une conversion préalable vers PNG 16-bit, TIFF ou autre format compatible
- Impact potentiel sur la qualité et la taille des fichiers

5.2 LOD et précision des UV

- La simplification automatique peut altérer légèrement les coordonnées UV
- Problème accentué avec des textures très haute résolution
- Les normales peuvent être recalculées de manière imprécise
- Support partiel des submeshes (détection des bordures limitée)

5.3 Architecture du système LOD

- **Contrairement au format Nexus (.nxz), Babylon.js ne charge pas un fichier unique avec LOD intégré**
- Nécessite soit plusieurs fichiers pré-générés, soit génération en temps réel
- La génération en temps réel peut prendre du temps pour des meshes complexes
- Consommation mémoire: tous les niveaux LOD sont gardés en mémoire simultanément

6. Recommandations

6.1 Pour le support des textures .EXR

- **Option 1 (recommandée):** Convertir les fichiers .EXR en PNG 16-bit avant le chargement. Utiliser un pipeline de conversion en amont.
- **Option 2:** Développer un loader custom EXR en JavaScript (complexe, nécessite une bibliothèque comme three.js EXRLoader adaptée)
- **Option 3:** Envisager un format alternatif (TIFF 16-bit, DDS) si la compatibilité navigateur le permet

6.2 Pour le système LOD avec .OBJ

- **Option 1 (qualité maximale):** Pré-générer les niveaux LOD dans notre logiciel 3D (Blender, Maya) et charger plusieurs fichiers .OBJ
- **Option 2 (automatique):** Utiliser l'Auto-LOD de Babylon.js, mais tester rigoureusement avec nos textures haute résolution pour vérifier les artefacts UV
- **Option 3 (hybride):** Créer 2-3 niveaux manuellement pour les distances proches/moyennes, utiliser l'Auto-LOD pour les distances lointaines

6.3 Stratégie globale

- Effectuer des tests de performance avec nos modèles réels
- Mesurer l'impact mémoire GPU avec des textures haute résolution
- Documenter les limites acceptables de simplification pour notre cas d'usage

7. Conclusion

Babylon.js v8.50.2 offre un support robuste pour les fichiers .OBJ et un système LOD fonctionnel, mais présente des limitations importantes pour notre cas d'usage spécifique:

- **L'absence de support natif .EXR** nécessite une étape de conversion préalable qui peut impacter la qualité et le workflow.
- **Le système LOD fonctionne par fichiers multiples** plutôt que par fichier unique avec décimation dynamique, ce qui diffère de l'approche Nexus.
- **L'Auto-LOD peut introduire des imprécisions UV** potentiellement problématiques avec des textures très haute résolution.

Pour un projet nécessitant des fichiers .OBJ avec textures .EXR et système LOD, il sera nécessaire de:

- Mettre en place un pipeline de conversion de textures
- Choisir entre LOD pré-généré (qualité) ou automatique (praticité)
- Effectuer des tests approfondis de performance et qualité visuelle

Le support Gaussian Splatting (.PLY) est excellent et mature dans Babylon.js 8.x, offrant une alternative intéressante si notre workflow peut s'adapter à cette technologie de capture 3D.

8. Références

- Documentation Gaussian Splatting:
<https://doc.babylonjs.com/features/featuresDeepDive/mesh/gaussianSplatting>
- Documentation LOD:
<https://doc.babylonjs.com/features/featuresDeepDive/mesh/LOD>
- Documentation Simplification:
<https://doc.babylonjs.com/features/featuresDeepDive/mesh/simplifyingMeshes>
- Documentation OBJ Loader:
<https://doc.babylonjs.com/features/featuresDeepDive/importers/oBJ>